Deduce, Propose, Correct: Probability-Accountable LLM Shortlists for Typed Program Synthesis

Tri Nguyen

Thanh-Dat Nguyen
Harvard University
Basis Research Institute
datnguyen@seas.harvard.edu

Abstract

Large language models can guide program synthesis, but importance-corrected sequential Monte Carlo requires an evaluable proposal probability for every sampled program. We present DPC, a typed programming-by-example system in which a model supplies four repair slots and the application converts the resulting shortlist into a full-support proposal over a bounded grammar. In a fresh-blind study on 12 finite-domain filter–map tasks, the LLM-guided system solved 10 tasks under a 29-slot budget, versus 2 for matched grammar-random acquisition; all eight discordant pairs favored LLM guidance ($p = 1/256$, one-sided exact sign test). This discovery result did not establish calibration: provider-free V1 and a later 20-task fresh V2 study both failed their exact-program-mass gates. After a reused-task diagnostic, a redesigned factorized proposal–bridge method was frozen and tested on a second fresh suite of 32 tasks. At $N = 256$, it passed every gate (RMSE 0.0399; bootstrap upper bound 0.0494; bias -0.0109, central 90% interval $[-0.0150, -0.00764]$), and RMSE fell to 0.0259 at $N = 1024$. This confirmation concerns only provider-free terminal finite-support inference on singleton-complete synthetic tasks; it does not calibrate LLM guidance, four-stage SMC, the full PBE system, or target mean loss.

Keywords: program synthesis, programming by example, large language models, sequential Monte Carlo, neuro-symbolic inference

1 Introduction

Programming by example (PBE) asks for a program that maps each supplied input to its paired output. The specification is accessible but incomplete: many programs may fit a small example set, and the space of typed recursive programs grows combinatorially. Classical systems control that space with domain languages, deduction, and structural search orders (Gulwani, 2011; Feser et al., 2015); learned policies and LLMs can instead prioritize plausible code (Kalyan et al., 2018; Barke et al., 2024; Li et al., 2024). That learned guidance changes the search order, but it also makes the source of any improvement harder to identify.

That shift creates an attribution problem. A type checker, execution engine, or selector may determine success even when the model only proposes candidates. Reporting only the selected program also conceals invalid responses, duplicate proposals, and the rules used to choose among them. We therefore evaluate the complete model–interpreter–SMC system while stating its proposal, target, budget, and comparator separately.

Sequential Monte Carlo (SMC) makes that accounting requirement precise. Programs become particles, semantic fit defines stage targets, and resampling concentrates computation. This view has precedents in neural program synthesis (Ellis et al., 2019) and executable model discovery (Wahl et al., 2026). Its probabilistic interpretation, however, requires an evaluable proposal probability. A free-form response does not by itself provide the probability of the canonical abstract syntax tree consumed by an interpreter, especially when invalid responses, duplicate repairs, and textual aliases are present.

DPC addresses this gap without treating model output as a program probability. The model supplies four typed repair slots, and the application turns them into the distribution used for sampling and weighting. Treating the response as an auxiliary variable removes the unknown provider-response law from the importance weight. The resulting transition is fully evaluable without being presented as an LLM distribution over programs.

The methodological contribution is a bounded synthesis mechanism whose proposal can be reconstructed from the recorded shortlist. It combines typed execution evidence, fixed four-slot totalization, a normalized grammar restart, explicit SMC weights, and a strict boundary between online search and exact reference computation. Together, these components turn each recorded shortlist into a replayable proposal.

We evaluate that proposal through distinct evidentiary layers. A fresh-blind paired study asks whether LLM shortlists improve bounded discovery. A provider-free calibration sequence then preserves two failed proposals, uses post-failure diagnostics to redesign the proposal, and tests the redesign on a second independently generated suite. Finally, a released-data ExeDec adapter exercise provides a limited stress test. This ordering keeps the central distinction visible: discovery can improve while distributional accuracy depends on the proposal, estimator, target functional, and study domain.

2 Related Work

DPC lies at the intersection of symbolic PBE, learned search guidance, and particle inference. Symbolic PBE systems use domain languages, deduction, and structural biases to control ambiguity (Gulwani, 2011; Feser et al., 2015). Neural guided deduction and LLM-guided synthesis instead learn or prompt search priorities (Kalyan et al., 2018; Barke et al., 2024; Li et al., 2024). Our narrower focus is the sampling law created when a learned system proposes a finite repair shortlist.

SMC and related particle methods provide standard tools for sequential importance weighting and resampling (Del Moral, 2004; Del Moral et al., 2006; Naesseth et al., 2019), and prior synthesis systems have combined learned guidance with particle inference (Ellis et al., 2019; Wahl et al., 2026). The contribution here is not the use of SMC itself. We focus on the totalized four-slot proposal and use exact references to show both that evaluable proposal probabilities do not by themselves guarantee finite-N calibration and that a factorized redesign can pass a separately frozen terminal finite-support gate.

ExeDec studies execution decomposition across released DeepCoder and RobustFill tasks (Shi et al., 2024). Our adapter neither implements the complete DeepCoder language nor reproduces ExeDec's official split structure. We use it only as a released-data stress test after defining the much narrower typed filter–map setting below.

3 Typed Filter–Map Synthesis

Let $\mathcal{D} = \{(x_j, y_j)\}_{j=1}^{M}$ be examples with inferred signature $\tau_{\text{in}} \to \tau_{\text{out}}$. A program e is exact on $\mathcal{D}$ when $e(x_j) = y_j$ for every j. The bounded typed DSL contains integer and Boolean expressions, lists, conditionals, `map`, and right folds. A structural cost favors compact programs, following the minimum-cost perspective of Lambda^2 (Feser et al., 2015). Exactness on examples is not, by itself, evidence of semantics outside the declared evaluation domain, so every study states that domain explicitly.

The confirmatory study restricts programs to the following filter–map skeleton:

$$\text{foldr}(\lambda i, a.\ \textbf{if}\ p(i)\ \textbf{then}\ g_m(i) :: a\ \textbf{else}\ a,\ [],\ x), \tag{1}$$

with typed holes $p : \text{Int} \to \text{Bool}$ and $g_m : \text{Int} \to \text{Int}$. A normalized recursive grammar defines a law $g(e)$ over complete programs within the declared bounds. The provider need not know or enumerate that complete-program support.

Examples constrain individual holes only when their effects can be isolated. A singleton example may establish, for instance, that $p(-3) = \text{false}$ or $g_m(2) = 4$; an ambiguous example yields no hole-level fact. For each current program, the harness records predicate and mapper violations, execution loss, the selected typed hole, and the legal local grammar. Only this public execution evidence is supplied to the provider.

All validity decisions remain with the application. It parses and type-checks candidate syntax, executes programs under fixed resource limits, and canonicalizes states before caching. The provider proposes repairs but cannot certify their validity or success. This separation lets the grammar-random arm share the interpreter, evidence, target, weighting, and resampling code. With validity and execution fixed by the application, the remaining problem is to define the law from which repairs are actually sampled.

4 Probability-Accountable Repair Shortlists

Given current program x, selected hole h, and structured public evidence, the provider is asked for four typed repairs. Every missing, malformed, invalid, or no-op slot is mapped to x, and duplicate multiplicity is retained. If the four totalized slots are $S = (s_1, \ldots, s_4)$, the application defines

$$H_S(y) = \frac{1}{4}\sum_{j=1}^{4} \mathbf{1}[s_j = y] \quad \text{and} \quad q(y \mid S, x, h) = (1-\varepsilon)H_S(y) + \varepsilon g(y). \tag{2}$$

Because g is normalized and has full support over the declared bounded grammar, q is normalized, evaluable, and positive on every grammar program. We neither retry nor backfill a failed response; it contributes four copies of the current state and preserves the fixed denominator. The r5, V1, ExeDec V2, and sticky V2 arms use $\varepsilon = 0.05$, while terminal V2 also evaluates prespecified diagnostic alternatives.

We treat the raw response as an auxiliary variable, not as a probability distribution over programs. Let $R_t^a(S_t \mid x_{t-1}, h_t, \mathcal{D})$ be the shortlist law for acquisition arm a. This law is unknown for the LLM arm, weighted sampling without replacement among legal non-current repairs for grammar-random, and a point mass for the deterministic V1 ranking

oracle. At an exact V1 parent, acquisition is skipped and the shortlist contains four copies of the current program, although the grammar restart still gives the overall proposal full support.

The same factor R_t^a appears in the extended target and proposal, so it cancels from the importance weight. The four-slot arms are then weighted with Equation 2. Terminal V2 uses separately specified proposal laws, including a 64-slot empirical distribution for its top-64 control. This cancellation accounts for the transition actually used without claiming to recover the model's marginal probability for a program.

An evaluable sampling law need not be an efficient importance proposal. If the shortlist places little mass where the target does, the weights may have high variance even when every proposal probability is correct. The next section uses this explicit law as the denominator in the particle correction. The studies then evaluate exact-program discovery and target-mass approximation as separate endpoints.

5 Importance-Corrected SMC

The stage targets combine grammar mass, execution evidence, and program cost. Let $V_p(x)$ and $V_m(x)$ be fixed predicate- and mapper-singleton violation counts, let $L_{\mathcal{D}}(x)$ be capped soft execution loss, and let $C(x)$ be program cost. The r5 and V1/V2 tasks use $(\lambda_L, \lambda_C, \lambda_E) = (0.75, 0.02, 2)$; ExeDec V2 uses its frozen scorer defaults $(2.0, 0.15, 2)$. Write $\ell(x) = -\lambda_L L_{\mathcal{D}}(x) - \lambda_C C(x)$. The four unnormalized single-program stage densities are

$$\begin{aligned}
\gamma_1(x) &= g(x)\exp[-\lambda_E V_p(x)],\\
\gamma_2(x) &= g(x)\exp[-\lambda_E\{V_p(x) + V_m(x)\}],\\
\gamma_3(x) &= g(x)\exp[-\tfrac{\lambda_E}{2}\{V_p(x) + V_m(x)\} + \tfrac{1}{2}\ell(x)],\\
\gamma_4(x) &= g(x)\exp[\ell(x)].
\end{aligned} \tag{3}$$

These densities define finite computational targets. The soft-loss exponential is not a real-world likelihood or a posterior over unbounded code.

We can now write the cancellation on an extended state space that includes the realized shortlists. Conditioning on the initial program x_0, arm a has target and proposal

$$\begin{aligned}
\Gamma_t^a(x_{1:t}, S_{1:t}) &= \prod_{s=1}^{t} R_s^a(S_s \mid x_{s-1}, h_s, \mathcal{D})\,\gamma_s(x_s),\\
Q_t^a(x_{1:t}, S_{1:t}) &= \prod_{s=1}^{t} R_s^a(S_s \mid x_{s-1}, h_s, \mathcal{D})\,q(x_s \mid S_s, x_{s-1}, h_s).
\end{aligned} \tag{4}$$

The shared R_s^a terms cancel, leaving incremental potential

$$G_t(x_{t-1}, S_t, x_t) = \frac{\gamma_t(x_t)}{q(x_t \mid S_t, x_{t-1}, h_t)}. \tag{5}$$

The realized shortlist still affects q, but evaluating the marginal free-form LLM probability of x_t is unnecessary.

The particle implementation applies this potential to every scheduled child. If parent i has normalized weight W_{t-1}^i and produces $B_{t,i}$ children, child b receives unnormalized weight

$$\widetilde{W}_t^{i,b} = \frac{W_{t-1}^i}{B_{t,i}} \frac{\gamma_t(X_t^{i,b})}{q(X_t^{i,b} \mid S_t^i, X_{t-1}^i, h_t^i)}. \tag{6}$$

The factor $1/B_{t,i}$ allocates parent mass across scheduled offspring. After stages 1–3, systematic resampling selects the frozen parent count and resets retained weights. Marginalizing auxiliary shortlists and previous states gives terminal marginal γ_4/Z_4; the product-path normalizer is a different quantity. The calibration endpoint below concerns the terminal target, not that path normalizer.

The studies also separate online search from exact reference computation. The r5 and ExeDec V2 online accounts cover only scheduled complete-program executions and report logical slots, online physical scorer calls, and provider calls; r5 also reports cache reuse and tokens. The provider-free calibration studies and ExeDec V2's later reference step enumerate bounded supports separately. Those evaluations are excluded from online work, and programs found there are never credited as online discoveries.

6 Studies

The evaluation begins with bounded discovery: r5 compares the complete LLM-shortlist system with a matched grammar-shortlist system. Calibration V1 then asks whether the weighted particles approximate an exactly enumerable target. The calibration record continues through a conditioned terminal diagnostic, a failed fresh V2 confirmation, a reused-task factorized diagnostic, and a second fresh V3 confirmation. ExeDec V2 is reported last as a limited stress test of the released-data adapter; none of these endpoints is pooled with the r5 discovery result.

6.1 Fresh-blind r5 paired discovery

Before task generation, the r5 method and analysis plan were frozen. A new custodial secret initialized a seeded generator, which then drew 12 independent targets from the rejection-conditioned recursive filter–map grammar. Each public task contained every singleton input in the declared integer domain $\{-3, \ldots, 4\}$ together with list examples.

The provider was GPT-OSS-120B at a sealed revision (OpenAI, 2025). Its settings, also used in ExeDec V2, were temperature zero, low reasoning effort, a 1,200-token output cap, a 420-second timeout, concurrency two, and no retries. Private target material stayed outside the provider environment until public execution, reveal-free analysis, and public sealing had finished.

For each task, grammar-random ran before the LLM arm. Both used the same initial law, evidence, hole policy, four-slot totalization, grammar restart, stage targets, importance weights, resampling, and 29-slot trajectory. They differed only in shortlist acquisition. Task-specific start, proposal-mixture sampling, and resampling seeds were paired; only acquisition seeds differed. The paired design therefore compares shortlist-acquisition mechanisms within the complete shared system.

Each trajectory began with one executed program. Stage 1 drew four children, and stages 2–4 drew four children from each of two resampled parents, yielding $1+4+3\times 8 = 29$ logical slots and eight terminal weighted particles. The primary endpoint was zero public-example loss by slot 29. Complete singleton coverage made this endpoint equivalent to itemwise filter–map behavior on the declared domain, a relationship checked only after unblinding. The task was the statistical unit, and the analysis used a one-sided exact sign test over discordant outcomes. Every trajectory ran the full budget without outcome-dependent stopping.

6.2 Provider-free calibration sequence

Calibration V1 was frozen separately and used no provider. Four developmental filter–map tasks each had an exactly enumerable 36,000-program support. A deterministic public-evidence oracle ranked legal local repairs exhaustively and supplied four slots, which were mixed with the same grammar restart. This oracle was a diagnostic component, not an LLM or a search baseline.

The study used $N \in \{32, 64, 128, 256, 512\}$ terminal weighted particles and 32 repetitions for each task and value of N. Each repetition scheduled $1 + 4N$ logical slots and did not stop early. The frozen $N = 256$ gate required pooled exact-program-mass RMSE below 0.10 and signed bias inside $[-0.03, 0.03]$.

Because V1 failed that gate, terminal diagnostic V2 was frozen before any new Monte Carlo draws. It checked the finite-state importance identities and compared the sticky V1 terminal proposal with local alternatives and a global top-64, $\varepsilon = 0.50$ positive control selected after exhaustive support scoring. It evaluated six arms at $N \in \{256, 512\}$ with 128 repetitions per task–arm–N cell. The diagnostic conditions on a mechanically selected zero-loss parent and addresses only the terminal calculation. It leaves V1's decision unchanged and does not test the four-stage recurrence or online search efficiency.

Fresh calibration V2 asked the same terminal exact-mass question on 20 newly generated tasks, each with a 36,000-program support and complete singleton evidence. Its provider-free proposal–bridge method selected eight semantic modes by ranking a fixed grammar sample, assigned 0.25 mass to the grammar and 0.75 to local neighborhoods, and used no primary Metropolis–Hastings moves. The primary $N = 256$ cell and descriptive $N \in \{512, 1024\}$ cells each used 128 repetitions per task; task-first bootstrap intervals used 20,000 replicates. All thresholds, seeds, source bindings, task custody, and the rule for publishing failure were frozen before the fresh secret.

After V2 failed, a factorized diagnostic changed only mode acquisition. It ranked predicate and mapper behavior classes separately from public singleton evidence, crossed the top eight of each, scored at most 64 complete representatives, and retained eight semantic modes. The diagnostic reran the frozen particle counts and seeds on the same 20 V2 tasks, so its numerical success could motivate a new method but could not replace the failed V2 gate.

Fresh V3 R2 then tested that factorized method on a second independently generated suite of 32 tasks. Before any exact reference was materialized, the study acquired all 32 public mode banks; the primary bridge followed $q(p)^{1-\beta}\gamma(p)^{\beta}$ with adaptive steps, no primary Metropolis–Hastings moves, and $N \in \{256, 512, 1024\}$. Each cell used 64 repetitions

with equal task weight. Matched factorized terminal importance sampling and the historical sticky proposal at $N = 256$ were descriptive comparators. The frozen gate jointly required a task-first 95% RMSE upper bound below 0.10, a central 90% bias interval strictly inside $(-0.03, 0.03)$, every task RMSE below 0.20, nonincreasing point RMSE, controlled weight tails, and a finite-state identity error at most 10^{-12}.

6.3 ExeDec V2 released-data debug

ExeDec V2 is a descriptive debug study over four deduplicated targets from the released ExeDec DeepCoder data (Shi et al., 2024). The local adapter accepts only a strict Filter-then-Map subset and collapses each two-line source program to the skeleton in Equation 1; "DeepCoder-HO" is a local label, not an official ExeDec split. The design pairs eight seeds per task across LLM-SMC and grammar-random, giving 32 paired blocks and 64 scheduled runs. Its endpoint is exactness on finite private debug probes, with an exact two-sided discordant-pair test and no efficacy gate.

The targets and examples are public and may have appeared in model training data. The private probes are finite debug checks, not evidence of fresh generalization or a proof of semantic equivalence. In addition, the shared-FUSE staging tree did not provide exclusive operating-system custody, and a separate operations-evidence export failed its packaging verifier. We therefore treat ExeDec V2 only as an adapter debug study. Appendix C records the custody and packaging limitations.

7 Results

The r5 study supports bounded discovery, while the calibration sequence shows that accuracy is proposal-specific: V1 and fresh V2 failed, and redesigned fresh V3 passed a narrower terminal finite-support gate. The ExeDec stress test is reported separately because its released data and repeated-seed design give it a different evidential status.

7.1 Fresh-blind r5 discovery

The r5 LLM arm found a finite-domain exact program by slot 29 on 10 of 12 tasks, compared with 2 of 12 for grammar-random. All eight discordant tasks favored the LLM arm, yielding a one-sided exact sign-test value of $p = 1/256 = 0.00390625$. The prespecified requirements of at least six LLM successes and a four-task advantage were both met. At the descriptive slot-21 checkpoint, the two arms had solved 8 and 2 tasks. These results compare complete systems that differ only in shortlist acquisition; they do not isolate the model from the shared interpreter, weighting, or resampling machinery.

The 24 trajectories executed 696 logical program slots. Scorer caching reduced this to 367 physical scorer calls, with 329 cache hits. The LLM arm made 58 provider calls and used 137,759 tokens; grammar-random made no provider calls. Two responses ended at the 1,200-token limit and, as frozen, each was totalized to four current-state sentinel slots without retry or backfill.

The post-unblind audit verified the secret and target commitments, reproduced all 12 public tasks, and re-executed every hidden target. It ran only after reveal-free analysis and a 415-file public checksum inventory had been sealed. Across the 12 successful task–arm

trajectories, four first-exact programs were syntax-identical to the target and eight were semantic aliases. Because public singletons covered the complete declared domain, this is finite-domain semantic recovery, not unique-AST or out-of-domain recovery.

7.2 Calibration failures, diagnosis, and fresh V3 confirmation

V1 failed both frozen $N = 256$ gate components despite exact-program discovery in all 128 task–repetition runs (Table 1). The result shows that discovering an exact program did not answer the calibration question.

Table 1: Frozen provider-free V1 calibration gate at $N = 256$, pooled over four tasks and 32 repetitions per task. Discovery was descriptive.

Endpoint	Criterion	Observed	Decision
Exact-mass RMSE	< 0.10	0.2395265	Fail
Signed exact-mass bias	$[-0.03, 0.03]$	$+0.1130184$	Fail
Zero-loss discovery	descriptive	128/128	Not gated

For $N = 32, 64, 128, 256, 512$, pooled exact-mass RMSE was 0.32468, 0.26776, 0.26536, 0.23953, and 0.25592, respectively. Across this grid, RMSE did not decrease at the reference $N^{-1/2}$ rate. Exact evaluation of q was necessary for correction, but it was insufficient to meet the finite-budget target under this proposal, schedule, and four-task suite.

Terminal V2 first verified the finite-state importance identities to a maximum absolute error of 2.22×10^{-16}. The positive control used $\varepsilon = 0.50$ and a global top-64 shortlist selected by exhaustively scoring the public support. It increased the exact population ESS fraction on every task and reduced pooled $N = 512$ exact-mass RMSE from 0.2792 to 0.0626 relative to the sticky proposal. This met the prespecified mechanism-support rule and implicated poor proposal–target overlap in the conditioned terminal step. Because the control scores the complete finite support and starts from a zero-loss parent, it is a diagnostic rather than a practical search method and leaves the V1 failure unchanged.

Fresh V2 also failed its frozen gate. At $N = 256$, the primary bridge had exact-mass RMSE 0.2612355 and bias -0.0748415; the task-first bootstrap 95% upper bound on RMSE was 0.3767653, and its central 90% bias interval was $[-0.1497656, -0.0061850]$. Although pooled RMSE decreased to 0.2225051 and 0.1458710 at $N = 512$ and 1024, respectively, two task-level RMSE values exceeded 0.82. These retained outcomes establish a second calibration failure, not a reason to replace difficult tasks.

The post-failure factorized diagnostic on those same 20 tasks reduced the $N = 256$ RMSE to 0.0221995; its bootstrap upper bound was 0.0245201 and its central 90% bias interval was $[-0.0081112, -0.0057479]$. Because the factorized acquisition rule was designed after observing V2 and reused its tasks and seeds, this result is mechanism evidence only. It justified the second fresh test but supplied no confirmatory outcome itself.

Fresh V3 passed all six components of its frozen primary gate. At $N = 256$, the factorized proposal–bridge estimate had bias -0.0109472 and RMSE 0.0398662; the task-first bootstrap RMSE upper bound was 0.0494323 and the central 90% bias interval was

$[-0.0149663, -0.00763969]$. The largest task-level RMSE was 0.102853, and point RMSE decreased from 0.0398662 to 0.0304513 to 0.0258555 over $N = 256, 512, 1024$. Table 2 records the complete gate rather than selecting only its favorable components.

Table 2: Fresh V3 R2 provider-free terminal exact-mass calibration gate on 32 newly generated tasks with 64 repetitions per task–cell. Every row was required to pass.

Gate component	Criterion	Observed
Task-first RMSE upper 95%	< 0.10	0.0494323
Task-first bias central 90%	inside $(-0.03, 0.03)$	$[-0.0149663, -0.00763969]$
Maximum task RMSE at $N = 256$	< 0.20	0.102853
Point RMSE at $N = 256, 512, 1024$	nonincreasing	0.0398662, 0.0304513, 0.0258555
95th percentile of runwise max weight	< 0.05	0.0117401
Fraction of max weights above 0.25	at most 0.01	0
Maximum identity error	at most 1e-12	1.33e-15

The descriptive $N = 256$ matched factorized terminal-importance arm had exact-mass RMSE 0.0421178, while the historical sticky arm had RMSE 0.110263. These comparisons help locate the gain but were not confirmation gates. More importantly, the primary method's target-mean-loss RMSE was 0.2991 at $N = 256$, so the pass applies to exact-program target mass, not to every posterior functional. It is evidence for a provider-free terminal estimator on this singleton-complete finite law, not for LLM calibration, the four-stage recurrence, or full-PBE inference.

7.3 ExeDec V2 debug

All 64 runs formed 32 complete paired seed blocks. LLM-SMC was exact on every stored private debug probe in 17/32 blocks, versus 3/32 for grammar-random, a success-fraction difference of 0.4375. There were 16 LLM-only successes, two grammar-random-only successes, one success in both arms, and 13 in neither arm, so the 18 discordant blocks gave an exact two-sided conditional sign-test value of $p = 0.001312255859375$. We treat this comparison as descriptive because the eight seeds per task are repeated measurements on only four released targets and the protocol specified no efficacy gate. This exact calculation is not evidence of statistical significance or confirmation.

Public-example exactness was 21/32 for LLM-SMC and 5/32 for grammar-random. The runs used 1,856 logical program slots and 1,040 online physical scorer calls before reference enumeration; exhaustive public-reference evaluations are excluded from that count. LLM-SMC made 188 provider calls under the frozen aggregate cap of 224; grammar-random made none.

Among runs that passed the private probes, the mean first-success slot was 12 for LLM-SMC and 9 for grammar-random. Because this summary conditions on success, it is not a speed comparison. Mean terminal ESS out of eight particles was 4.7512 and 3.0567, respectively, and passing finitely many probes does not establish semantic equivalence.

Table 3: ExeDec V2 private-debug exact successes by released target. Each row contains eight repeated paired seed blocks; these four targets, not 32 fresh tasks, define the limited target diversity.

Released target	LLM-SMC	Grammar-random
`exedec-debug-0131`	2/8	0/8
`exedec-debug-0245`	5/8	1/8
`exedec-debug-0258`	4/8	1/8
`exedec-debug-0608`	6/8	1/8
Overall	17/32	3/32

8 Limitations and Integrity

The r5 result covers a narrow setting: 12 generated filter–map tasks over eight integer values, with public singleton examples that identify behavior on that domain, one pinned model revision, and one matched acquisition comparator. It does not support extrapolation beyond the observed domain or represent arbitrary PBE, and temperature-zero calls are not independent model replicates.

Calibration remains deliberately narrow. V1 and fresh V2 retain failed gates; their post-failure diagnostics do not revise those decisions. Fresh V3 confirms one redesigned provider-free proposal–bridge estimator for terminal exact-program mass on a singleton-complete synthetic law, but it does not test the LLM shortlist, the four-stage recurrence, or a large DSL. Its target-mean-loss error also shows that a pass for one functional is not a pass for every posterior quantity. Exact references were possible only because each bounded support contained 36,000 programs.

The ExeDec result has narrower evidential value than r5. It uses four deduplicated released targets, eight repeated seeds within each target, an unofficial strict Filter-then-Map adapter, potentially contaminated public data, and finite debug probes. Its 32 blocks are not 32 independent targets, and its exact test value is descriptive rather than confirmatory. The scorer scales also differ from r5. Nonexclusive FUSE staging and the failed original operations-evidence safe-mode verification impose a further integrity limit that the noncanonical local derivative does not remove.

Across the studies, equal logical-slot budgets do not imply equal wall time, provider cost, or total computation. We therefore make no speedup claim against enumeration or other synthesizers. Generated programs also require sandboxed execution and human review. Appendix C records the failed and superseded protocols relevant to these limits.

9 Conclusion

DPC turns four typed LLM repair slots into an application-computed, full-support proposal law over the declared bounded grammar and uses that law in an explicit SMC correction. In the fresh-blind study, the complete LLM-shortlist mechanism solved 10 of 12 tasks versus 2 of 12 for matched grammar acquisition. This supports a bounded discovery advantage for

the complete tested system. It does not isolate an LLM-only effect or establish a speedup or broad generalization.

The calibration sequence qualifies that discovery result rather than reversing it. V1 and fresh V2 show that evaluable probabilities and exact-program discovery do not guarantee accurate target-mass estimates; the factorized redesign then passes a second fresh test for one terminal finite-support functional. Because target mean loss remained much less accurate and neither the provider nor the four-stage recurrence was tested, the next step is a new full-pipeline calibration study followed by independently selected external tasks. Discovery, distributional accuracy by functional, and computational cost should remain separate endpoints.

Acknowledgments and Disclosure of Funding

This draft received no declared external funding. Compute and hardware metadata remain under author review. The final submission must include complete funding, compute, competing-interest, and author disclosures.

Appendix A. Proposal Normalization

For the four-slot method arms, totalization guarantees that S has exactly four entries. Thus H_S in Equation 2 sums to one even if every provider slot fails or all four entries are duplicates. The recursive grammar law g also sums to one, so their convex mixture is normalized; when $\varepsilon > 0$, every program in the declared grammar receives positive mass. The V2 top-64 control uses the analogous normalized 64-slot empirical law. In the extended target and proposal, the same normalized acquisition factor R_t^a appears at each stage and cancels from Equation 5. This proves normalization of the implemented proposal laws, not finite-N calibration.

The fresh proposal–bridge studies use a separate finite-support law. It assigns 0.25 total mass to the normalized grammar and divides the remaining 0.75 equally among eight semantic modes. Within a mode, the local kernel assigns 0.75 to its center and 0.125 each to predicate- and mapper-alias slices, with overlapping syntax masses added before normalization checks. The V3 artifact records this mass ledger and reconstructs every sampled $q(p)$; the bridge ratios telescope from q to the terminal target. This accounting makes the V3 proposal evaluable, while its calibration status still comes from the separate fresh empirical gate.

Appendix B. Task-Level Results

Table 4 reports every task–arm trajectory in the sealed r5 analysis. A provider-totalized slot is either an explicit no-op mapped to the current state or a sentinel produced by whole-response totalization; grammar-random made no provider calls.

Table 5 exposes the task heterogeneity behind the pooled V1 failure. RMSE and bias summarize 32 self-normalized exact-mass estimates for each task at $N = 256$.

Table 4: Fresh-blind r5 task-level outcomes. Success is zero public-example loss by slot 29; "–" means no exact discovery or no applicable provider quantity.

Task	Arm	Success	First exact	Best loss	Totalized slots
`blind-v3-01`	Grammar	No	–	12	–
`blind-v3-01`	LLM	Yes	29	0	1
`blind-v3-02`	Grammar	No	–	10	–
`blind-v3-02`	LLM	Yes	19	0	1
`blind-v3-03`	Grammar	No	–	11	–
`blind-v3-03`	LLM	Yes	22	0	5
`blind-v3-04`	Grammar	No	–	22	–
`blind-v3-04`	LLM	Yes	11	0	0
`blind-v3-05`	Grammar	No	–	17	–
`blind-v3-05`	LLM	Yes	14	0	0
`blind-v3-06`	Grammar	Yes	18	0	–
`blind-v3-06`	LLM	Yes	14	0	1
`blind-v3-07`	Grammar	No	–	20	–
`blind-v3-07`	LLM	No	–	20	2
`blind-v3-08`	Grammar	No	–	11	–
`blind-v3-08`	LLM	No	–	20	4
`blind-v3-09`	Grammar	No	–	15	–
`blind-v3-09`	LLM	Yes	6	0	0
`blind-v3-10`	Grammar	No	–	10	–
`blind-v3-10`	LLM	Yes	9	0	0
`blind-v3-11`	Grammar	No	–	12	–
`blind-v3-11`	LLM	Yes	9	0	0
`blind-v3-12`	Grammar	Yes	16	0	–
`blind-v3-12`	LLM	Yes	8	0	0

Table 5: Provider-free V1 calibration by task at $N = 256$. Discovery was descriptive and not a gate component.

Task	Reference mass	RMSE	Bias	Discovery	Mean ESS/N
`calibration-equality-scale`	0.1101251	0.2045285	−0.0501036	32/32	0.0173374
`calibration-lower-shift`	0.8988287	0.0960045	+0.0957645	32/32	0.9186058
`calibration-upper-negate`	0.9208953	0.0753309	+0.0662831	32/32	0.8776247
`foldr-bounded-square`	0.4944417	0.4156541	+0.3401296	32/32	0.4436003

Appendix C. Preserved Integrity and Failure Timeline

This appendix records protocol revisions that affected the status of the evidence. Blind-confirmation revisions r1 and r2 were superseded before secret preparation. R3 aborted before program execution or provider calls because a derived run record omitted two required fields. R4 completed public execution and reveal-free analysis but failed its sealing gate before unblinding, so it remains blinded and excluded. Its invalidated diagnostic was 11/12 versus 2/12 at slot 29 and 10/12 versus 0/12 at slot 21; these values disclose the revision path but are not evidence. Counting that diagnostic as a second numerical look gives the post hoc r5 Bonferroni value 0.0078125. Before any new secret or call, r5 changed only the sealer contract and focused test, retained r4's scientific execution method, and generated a fresh secret and suite.

The provider-free calibration sequence likewise retains its negative results. Calibration V1 failed and remains failed; terminal V2 was frozen afterward and is diagnostic only. Fresh V2 then failed on 20 newly generated tasks. The factorized V3 diagnostic was designed after that failure and reused the V2 tasks, so it had no confirmatory standing. Fresh V3 R1 was superseded before secret creation, custody, task generation, Monte Carlo draws, or provider calls solely to make the bias-interval wording match the already strict validator; it produced no numerical outcome. R2 used a new secret and independently generated 32 tasks, and its pass supplements rather than overwrites both failures.

Fresh V3 R2 acquired all 32 public mode banks before any exact reference was materialized. After execution, a deterministic replay regenerated every public bank, reference, seeded run, analysis value, and inventory binding without reading the private reveal. Unblinding occurred only after that replay passed; the final verification bound the protocol, method seal, custody seal, public manifest, analysis, inventory, replay receipt, and secret commitment. This sequence supports the freshness and reproducibility of the narrow V3 result, not a broader calibration claim.

The adapter studies have a separate revision history. ExeDec benchmark V1 was superseded before provider calls because its source closure omitted import-time files. ExeDec V2 is the final debug protocol. Its 1,143-file study archive passed the transfer verifier and was atomically imported, after which the canonical analysis passed its run-integrity checks. The separate original operations-evidence archive nevertheless failed its frozen safe-mode verifier at `files/operations/bin/exedec_v2_driver.py`. All 149 manifested content hashes and scientific/operational cross-bindings independently validated, but the shared-FUSE tar-header modes violated the sealed packaging contract, so the original failure remains authoritative.

A separately audited local derivative changed only the rejected archive's 150 tar mode headers to 146 files at `0600` and four executables at `0700`. That derivative and its report and seal are explicitly noncanonical and were not imported. Public-data, adapter, finite-probe, contamination, and nonexclusive-custody limits therefore apply to every report. We preserve the original archive, its failed verification, the noncanonical derivative, and the protocol history as separate records; none is rewritten to fit the final narrative.

Appendix D. Exact-Reference Claim Boundary

Exact reference enumeration serves two purposes: it determines whether exact programs exist and supplies target functionals for calibration. It does not measure synthesis speed. The V1 local ranking oracle and the V2 global top-64, $\varepsilon = 0.50$ control both use exhaustive public-score information that is unavailable to the online r5 search. Their discoveries, ranks, and work are not credited to the LLM or SMC search, and the positive control is not a practical synthesis algorithm.

Fresh V2 and V3 also use exhaustive support enumeration only to construct references and audit finite-state identities. In V3, factorized primary and matched-proposal acquisition completed for every task before any reference was materialized; only the descriptive historical sticky baseline was reference-derived. Exact references therefore make the calibration endpoint observable without turning the primary mode bank into an exhaustive-search oracle.

Appendix E. Artifact Availability

The data-and-paper companion is available in the public Hugging Face archive. Implementation source and tests are available from the GitHub publication branch; the completed-study source revision is `50369a6b0e264eda9cb6e1aab45949cb3be11cb6`.

R5 protocol SHA-256: `36bd14082c767f47327b73a1ee57f44763fcd97b2bea6dcc a0637a8cddc83d2d`.

R5 manifest: `artifacts/blind-filter-map-confirmation-v3-r5/ public-bundle-seal/SHA256SUMS`.

Calibration V1: `artifacts/particle-calibration-v1/analysis.json`.

Terminal diagnostic V2: `artifacts/particle-calibration-terminal-diagnostic-v2/analysis.json`.

Fresh calibration V2 failed analysis: `artifacts/calibrated-program-inference-v2-fresh/analysis.json`. SHA-256: `4d2fbca0a6a2f16f0db085b96c9eeec8dfd4938e453617107107462aff114008`.

Factorized reused-task diagnostic: `artifacts/ calibrated-program-inference-v3-factorized-diagnostic/analysis.json`. SHA-256: `31158c001ad67c6c95599fdd5c7ed9f85d1a619a01946f8bb6bff68b d9ee18b3`.

Fresh V3 R1 pre-secret supersession record: `papers/01-modelsmc/modelsmc-pbe-python/research/CALIBRATED_PROGRAM_ INFERENCE_V3_FRESH_R1_SUPERSESSION.md`. SHA-256: `295f8c40e046b281095d0d85 d8c04a25f5dc76fc02b335528f8b65b37209c3b9`.

Fresh V3 R2 protocol: `papers/01-modelsmc/modelsmc-pbe-python/research/ protocol-calibrated-program-inference-v3-fresh-r2.json`. SHA-256: `36f4a6c5930c2da2c89c1515d44267b6cc9bab94e7a000352f4343fd97fc97d6`.

Fresh V3 R2 method seal: `papers/01-modelsmc/modelsmc-pbe-python/research/ protocol-calibrated-program-inference-v3-fresh-r2.method-seal.json`. SHA-256: `788a44cc32ed617aa853a9e7ac97ab223684438285149eba0073cc6d 0a31dc4`.

Fresh V3 R2 custody seal:
artifacts/calibrated-program-inference-v3-fresh-custody-seal.json.
SHA-256: 32631befddaecd28aa4008c73232475b3676b26a7e1e0fe457dac699
5ab0932f.

Fresh V3 R2 public manifest: artifacts/
calibrated-program-inference-v3-fresh-suite/public/manifest.json.
SHA-256: 4fd1747c8e52b68a6fe14310ac39e178269c709303dec0c5cbed45b0
47f4cfc6.

Fresh V3 R2 analysis:
artifacts/calibrated-program-inference-v3-fresh/analysis.json. SHA-256:
51973aae5f443332973e65e3d60fa20ee328d60b12b478e1ed23d2430bb23e33.

Fresh V3 R2 inventory:
artifacts/calibrated-program-inference-v3-fresh/inventory.json. SHA-256:
bbac158dd44ab99d83f1cd404b2150aeaf4402fe9f75d694bb4bc0ed66995895.

Fresh V3 deterministic replay receipt: artifacts/
calibrated-program-inference-v3-fresh-replay-verification.json. SHA-256:
c7b08c0385867436e3e2d7a08cfe5a506ed8dfaea702defcabeff1e88b1859a7.

Fresh V3 unblind verification: artifacts/
calibrated-program-inference-v3-fresh-unblind-verification.json.
SHA-256: 3c3d5448800eaf85d904d05ea5e4387e8919c1dc1e1e4b4c7b5c8e97
57021868.

ExeDec V2 protocol SHA-256: 305cd2d89fbe0918a52b8e0ea6b3c4dfce23ad04
ae669c48bcc3fcaa37c15b7d.

ExeDec V2 imported study tree (1,143 files):
artifacts/exedec-deepcoder-ho-debug-benchmark-v2. Study archive SHA-256:
5e97831610852264f686d0f37d6f9c4aef8d783e45c29e6cc3fd5023dbc7e46c.

ExeDec V2 analysis:
artifacts/exedec-deepcoder-ho-debug-benchmark-v2/analysis/analysis.json.
SHA-256: 5bc9a74a39add1a60d8d5adedad89beb9dad389bd4e6cbdb4447b45e
1888c2bb.

ExeDec V2 analysis inventory: artifacts/
exedec-deepcoder-ho-debug-benchmark-v2/analysis/inventory.json. SHA-256:
848e204685c8b4459bb51c7471d5c644464f3183af142f6cecc60feb85a65b25.

Operations-evidence failure record: artifacts/
exedec-deepcoder-ho-debug-benchmark-v2-operations-evidence-failed-v1/
README.md. The rejected original archive and seal have SHA-256 values aa5b9fa4
a8b60ab318030cfb3ff98882de5889b5200a96ffb1e6ca85a8d96b0a and 72cc4bb3
998380eb12f78e15b7e17f04b6b676adf059b5efe6521ef804150ebe.

Noncanonical, unimported mode-header derivative archive/report/seal SHA-256:
cbb65b605b5212f521558249fe2036d8820b88056a4750ea280e5775e1f14393;
49e5f747b5dd78bde1e3a008205c139b5ad2ff8300602b3ede327db08af050d3; and
ebd74c77439879be78983bc98bfa7d5301cf30c7d3fb8ad4850f4d63926e3884. These
files remain under artifacts/
exedec-deepcoder-ho-debug-benchmark-v2-operations-evidence-failed-v1/
mode-header-normalized-derivative-v1.

The companion archive distributes data, checksums, failure records, and the manuscript; implementation source is distributed through the GitHub revision above. The path locators in this appendix are relative to that source checkout and its companion artifact tree.

References

Shraddha Barke, Emmanuel Anaya Gonzalez, Saketh Ram Kasibatla, Taylor Berg-Kirkpatrick, and Nadia Polikarpova. HYSYNTH: Context-free LLM approximation for guiding program synthesis. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://openreview.net/forum?id=5jt0ZSA6Co.

Pierre Del Moral. *Feynman-Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Springer, 2004. doi: 10.1007/978-1-4684-9393-1.

Pierre Del Moral, Arnaud Doucet, and Ajay Jasra. Sequential monte carlo samplers. *Journal of the Royal Statistical Society: Series B*, 68(3):411–436, 2006. doi: 10.1111/j.1467-9868.2006.00553.x.

Kevin Ellis, Maxwell Nye, Yewen Pu, Felix Sosa, Joshua Tenenbaum, and Armando Solar-Lezama. Write, execute, assess: Program synthesis with a REPL. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://arxiv.org/abs/1906.04604.

John K. Feser, Swarat Chaudhuri, and Isil Dillig. Synthesizing data structure transformations from input-output examples. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 229–239. ACM, 2015. doi: 10.1145/2737924.2737977.

Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 317–330. ACM, 2011. doi: 10.1145/1926385.1926423.

Ashwin Kalyan, Abhishek Mohta, Oleksandr Polozov, Dhruv Batra, Prateek Jain, and Sumit Gulwani. Neural-guided deductive search for real-time program synthesis from examples. In *International Conference on Learning Representations*, 2018. URL https://openreview.net/forum?id=rywDjg-RW.

Yixuan Li, Julian Parsert, and Elizabeth Polgreen. Guiding enumerative program synthesis with large language models. *arXiv preprint arXiv:2403.03997*, 2024. doi: 10.48550/arXiv.2403.03997.

Christian A. Naesseth, Fredrik Lindsten, and Thomas B. Schön. Elements of sequential monte carlo. *Foundations and Trends in Machine Learning*, 12(3):187–306, 2019. doi: 10.1561/2200000074.

OpenAI. gpt-oss-120b model documentation. OpenAI Developers documentation, 2025. URL https://developers.openai.com/api/docs/models/gpt-oss-120b. Accessed 2026-08-13.

Kensen Shi, Joey Hong, Yinlin Deng, Pengcheng Yin, Manzil Zaheer, and Charles Sutton. ExeDec: Execution decomposition for compositional generalization in neural program synthesis. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=oTRwljRgiv.

Stefan Wahl, Raphaela Schenk, Ali Farnoud, Jakob H. Macke, and Daniel Gedon. A probabilistic framework for LLM-based model discovery. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. URL https://arxiv.org/abs/2602.18266.